

Customer Shopping Behavior Analysis

End-to-End Data Analytics Project Using Python, PostgreSQL, SQL &
Power BI

Prepared by

Favaz Ahmed

B.Tech Computer Science & Engineering

PES University, Bengaluru

Technologies used

Python • Pandas • PostgreSQL • SQL • Power BI • GitHub

TABLE OF CONTENTS

Contents

Customer Shopping Behavior Analysis.....	1
End-to-End Data Analytics Project Using Python, PostgreSQL, SQL & Power BI.....	1
TABLE OF CONTENTS	2
1. Project Overview	3
2. Business Problem	4
3. Project Objectives.....	5
4. Dataset Description	6
Dataset Features.....	7
5. Technology Stack	8
6. Project Workflow.....	9
7. Data Preprocessing.....	10
8. Feature Engineering	13
9. PostgreSQL Database Integration	17
10. SQL Business Analysis.....	19
11.RFM-Style Customer Segmentation.....	24
12.Dashboard in Power BI.....	26
13. Key Insights.....	27
14. Business Recommendations	27
15. Conclusion	27

1. Project Overview

In today's competitive retail industry, organizations generate large volumes of customer transaction data every day. However, raw data alone does not provide meaningful insights unless it is properly processed, analysed, and visualized. Data analytics helps businesses understand customer purchasing behavior, identify sales trends, and support data-driven decision-making.

This project demonstrates a complete end-to-end data analytics workflow using **Python, PostgreSQL, SQL, and Power BI**. The primary objective is to analyse customer shopping behavior by transforming raw retail transaction data into valuable business insights.

The project begins with importing the customer shopping behavior dataset into Python for data exploration, cleaning, and preprocessing. After ensuring the data quality, the processed dataset is loaded into a PostgreSQL database. SQL is then used to answer important business questions related to customer spending patterns, subscription status, product categories, payment methods, and purchasing behavior. Finally, an interactive Power BI dashboard is developed to present the findings through meaningful visualizations that enable better business understanding.

This project demonstrates practical skills in data cleaning, exploratory data analysis (EDA), SQL querying, database management, and dashboard development, reflecting the typical workflow followed by data analysts in real-world business environments.

2. Business Problem

Retail businesses collect vast amounts of customer transaction data through both online and offline sales channels. This data contains valuable information about customer demographics, purchasing habits, preferred product categories, payment methods, and seasonal shopping trends. However, without proper analysis, organizations cannot effectively utilize this information to support strategic business decisions.

The objective of this project is to analyse customer shopping behavior and convert raw transactional data into meaningful business insights. By identifying spending patterns, customer preferences, subscription trends, and revenue-generating segments, businesses can make informed decisions related to marketing strategies, customer retention, inventory planning, and sales optimization.

This project applies Python for data preprocessing, PostgreSQL for data storage, SQL for business analysis, and Power BI for interactive visualization, providing a complete analytics solution that supports data-driven decision-making.

3. Project Objectives

The primary objectives of this project are:

- To analyse customer shopping behavior using retail transaction data.
- To perform data cleaning and preprocessing using Python and Pandas.
- To conduct Exploratory Data Analysis (EDA) to understand customer purchasing patterns.
- To apply feature engineering techniques for better data analysis.
- To import the processed dataset into a PostgreSQL database.
- To write SQL queries that answer real-world business questions.
- To develop an interactive Power BI dashboard for visualizing key business metrics.
- To derive actionable business insights and recommendations based on the analysis.
- To demonstrate an end-to-end data analytics workflow suitable for real-world business scenarios.

4. Dataset Description

Dataset Overview

The dataset used in this project is a **Customer Shopping Behavior Dataset** that contains retail transaction records and customer-related information. It includes demographic details, purchase history, shopping preferences, payment methods, seasonal trends, subscription status, and product information. The dataset is used to analyse customer purchasing behavior and generate business insights through Python, PostgreSQL, SQL, and Power BI.

Dataset Summary

Attribute	Details
Dataset Name	Customer Shopping Behavior Dataset
Domain	Retail Analytics
File Format	CSV
Number of Records	3,900
Number of Columns	18
Tools Used	Python, PostgreSQL, SQL, Power BI

Dataset Features

Feature	Description
Customer ID	Unique identifier assigned to each customer
Age	Age of the customer
Gender	Gender of the customer
Item Purchased	Product purchased by the customer
Category	Category of the purchased product
Purchase Amount (USD)	Total purchase amount in USD
Location	Customer's location
Size	Size of the purchased product
Color	Color of the purchased product
Season	Season in which the purchase was made
Review Rating	Customer's rating for the purchased product
Subscription Status	Indicates whether the customer has an active subscription
Payment Method	Method used to complete the payment
Shipping Type	Shipping option selected by the customer
Discount Applied	Indicates whether a discount was applied
Promo Code Used	Indicates whether a promotional code was used
Previous Purchases	Number of previous purchases made by the customer
Preferred Payment Method	Customer's preferred payment method
Frequency of Purchases	Frequency at which the customer makes purchases

5. Technology Stack

The successful completion of this project required the integration of multiple industry-standard tools and technologies. Each tool played a specific role in different stages of the data analytics workflow, ranging from data preprocessing and database management to business analysis and dashboard visualization.

Technology	Purpose
Python	Data loading, preprocessing, cleaning, and analysis
Pandas	Data manipulation and feature engineering
PostgreSQL	Storage and management of processed customer data
SQL	Business analysis and querying the database
SQLAlchemy	Connecting Python with PostgreSQL
Power BI	Building interactive dashboards and visualizations
Jupyter Notebook	Python development and documentation
GitHub	Version control and project portfolio hosting

Technology Justification

- **Python** was used to clean, preprocess, and prepare the dataset for analysis.
- **Pandas** simplified data manipulation and exploratory analysis.
- **PostgreSQL** provided a structured relational database to store the processed data.
- **SQL** was used to answer business questions and generate analytical insights.
- **Power BI** enabled the creation of an interactive dashboard for business users.
- **GitHub** was used to maintain project files and documentation while showcasing the project professionally.

6. Project Workflow

Raw CSV Dataset

|



Data Loading (Python)

|



Data Cleaning & Preprocessing

|



Exploratory Data Analysis (EDA)

|



Feature Engineering

|



Load into PostgreSQL

|



SQL Business Analysis

|



RFM-Style Customer Segmentation

|



Power BI Dashboard

|



Business Insights & Recommendations

7. Data Preprocessing

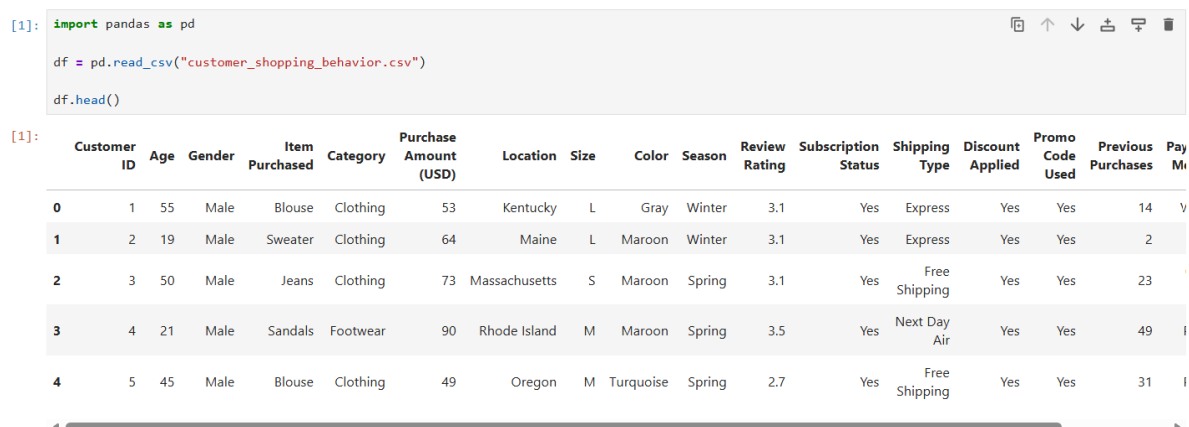
Introduction

Data preprocessing is a crucial step in any data analytics project, as the quality of analysis depends on the quality of the data. Before performing any business analysis, the dataset was examined, cleaned, and transformed to ensure consistency and accuracy. Python and the Pandas library were used to perform all preprocessing tasks.

Steps Performed

Include these bullet points:

- Imported the required Python libraries.
- Loaded the customer shopping behavior dataset into a Pandas DataFrame.
- Examined the dataset structure using `df.info()`.
- Displayed the first few records using `df.head()`.
- Checked for missing values.
- Verified duplicate records.
- Reviewed data types of all columns.
- Generated summary statistics using `df.describe()`.
- Prepared the dataset for feature engineering and database loading.



```
[1]: import pandas as pd
df = pd.read_csv("customer_shopping_behavior.csv")
df.head()
```

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Pay Me
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	Yes	14	V
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	Yes	2	
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	Yes	23	
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air	Yes	Yes	49	I
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	Yes	31	I

Figure 7.1: Loading and Previewing the Dataset

The dataset was successfully loaded into a Pandas DataFrame, and the first five records were displayed using `df.head()` to verify the data and its structure.

```
[2]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Customer ID                          3900 non-null   int64
1   Age                                  3900 non-null   int64
2   Gender                              3900 non-null   object
3   Item Purchased                      3900 non-null   object
4   Category                            3900 non-null   object
5   Purchase Amount (USD)               3900 non-null   int64
6   Location                            3900 non-null   object
7   Size                                3900 non-null   object
8   Color                               3900 non-null   object
9   Season                              3900 non-null   object
10  Review Rating                       3863 non-null   float64
11  Subscription Status                 3900 non-null   object
12  Shipping Type                       3900 non-null   object
13  Discount Applied                    3900 non-null   object
14  Promo Code Used                     3900 non-null   object
15  Previous Purchases                  3900 non-null   int64
16  Payment Method                     3900 non-null   object
17  Frequency of Purchases              3900 non-null   object
dtypes: float64(1), int64(4), object(13)
memory usage: 548.6+ KB
```

Figure 7.2: Dataset Structure and Information

The `df.info()` function was used to examine the dataset structure, including the number of records, data types, and non-null values for each column.

```
[3]: df.describe(include='all')
```

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900	3863.000000	3900	3900	3900	3900
unique	NaN	NaN	2	25	4	NaN	50	4	25	4	NaN	2	6	2	2
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring	NaN	No	Free Shipping	No	No
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999	NaN	2847	675	2223	2223
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN	3.750065	NaN	NaN	NaN	NaN
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN	0.716983	NaN	NaN	NaN	NaN
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN	2.500000	NaN	NaN	NaN	NaN
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN	3.100000	NaN	NaN	NaN	NaN
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN	3.800000	NaN	NaN	NaN	NaN
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN	4.400000	NaN	NaN	NaN	NaN
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN	5.000000	NaN	NaN	NaN	NaN

Figure 7.3: Descriptive Statistics of the Dataset

The `df.describe(include='all')` function was used to generate summary statistics for both numerical and categorical features, providing an overview of the dataset before further analysis.

```
[4]: df.isnull().sum()
```

```
[4]: Customer ID          0
     Age                0
     Gender             0
     Item Purchased      0
     Category           0
     Purchase Amount (USD) 0
     Location           0
     Size               0
     Color              0
     Season             0
     Review Rating      37
     Subscription Status 0
     Shipping Type      0
     Discount Applied   0
     Promo Code Used    0
     Previous Purchases 0
     Payment Method     0
     Frequency of Purchases 0
     dtype: int64
```

Figure 7.4: Missing Value Analysis

The dataset was checked for missing values using the `df.isnull().sum()` function. Identifying null values is an important preprocessing step, as missing data can affect the accuracy and reliability of the analysis.

8. Feature Engineering

Introduction

Feature engineering was performed to improve the dataset by creating new attributes and standardizing existing columns. These transformations made the dataset more suitable for SQL analysis and dashboard visualization.

```
[5]: df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median()))

[6]: df.isnull().sum()

[6]: Customer ID      0
Age                0
Gender             0
Item Purchased     0
Category           0
Purchase Amount (USD) 0
Location           0
Size               0
Color              0
Season             0
Review Rating      0
Subscription Status 0
Shipping Type      0
Discount Applied   0
Promo Code Used    0
Previous Purchases 0
Payment Method     0
Frequency of Purchases 0
dtype: int64
```

Figure 8.1: Missing values in the *Review Rating* column were replaced using the median rating of each product category.

```
[7]: df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(' ', '_')
df = df.rename(columns={'purchase_amount_(usd)': 'purchase_amount'})

[8]: df.columns

[8]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
        'purchase_amount', 'location', 'size', 'color', 'season',
        'review_rating', 'subscription_status', 'shipping_type',
        'discount_applied', 'promo_code_used', 'previous_purchases',
        'payment_method', 'frequency_of_purchases'],
        dtype='object')
```

Figure 8.2: Column names were standardized to snake_case for consistency and easier SQL querying.

```
[9]: #create a column age_group
labels = ['Young Adult', 'Adult', 'Middle-aged', 'Senior']
df['age_group'] = pd.qcut(df['age'], q=4, labels = labels)
```

```
[10]: df[['age', 'age_group']].head(10)
```

```
[10]:
```

	age	age_group
0	55	Middle-aged
1	19	Young Adult
2	50	Middle-aged
3	21	Young Adult
4	45	Middle-aged
5	46	Middle-aged
6	63	Senior
7	27	Young Adult
8	26	Young Adult
9	57	Middle-aged

Figure 8.3: Customer ages were categorized into age groups to simplify demographic analysis.

```
[11]: # create column purchase_frequency_days
frequency_mapping = {
    'Fortnightly': 14,
    'Weekly': 7,
    'Monthly': 30,
    'Quarterly': 90,
    'Bi-weekly': 14,
    'Annually': 365,
    'Every 3 Months': 90
}

df['purchase_frequency_days'] = df['frequency_of_purchases'].map(frequency_mapping)
```

```
df[['purchase_frequency_days', 'frequency_of_purchases']].head(10)
```

	purchase_frequency_days	frequency_of_purchases
0	14.0	Fortnightly
1	14.0	Fortnightly
2	7.0	Weekly
3	7.0	Weekly
4	365.0	Annually
5	7.0	Weekly
6	90.0	Quarterly
7	7.0	Weekly
8	365.0	Annually
9	90.0	Quarterly

Figure 8.4: Purchase frequency values were converted into equivalent days for easier analysis.

The engineered `purchase_frequency_days` feature was also used as a Recency proxy for customer segmentation. Since the dataset does not contain transaction dates or last-purchase timestamps, true Recency could not be calculated. Therefore, the estimated number of days between purchases was used as a documented proxy, where a lower number of days indicates more frequent purchasing behavior.

```
[13]: df[['discount_applied', 'promo_code_used']].head(10)
```

```
[13]:
```

	discount_applied	promo_code_used
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes
5	Yes	Yes
6	Yes	Yes
7	Yes	Yes
8	Yes	Yes
9	Yes	Yes

```
[14]: (df['discount_applied'] == df['promo_code_used']).all()
```

```
[14]: np.True_
```

```
[15]: df= df.drop('promo_code_used',axis=1)
```

```
[16]: df.columns
```

```
[16]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',  
        'purchase_amount', 'location', 'size', 'color', 'season',  
        'review_rating', 'subscription_status', 'shipping_type',  
        'discount_applied', 'previous_purchases', 'payment_method',  
        'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],  
        dtype='object')
```

Figure 8.5: The redundant promo_code_used column was removed after verifying it duplicated the discount_applied information.

9. PostgreSQL Database Integration

Introduction

After preprocessing, the cleaned dataset was transferred to a PostgreSQL database using SQLAlchemy. PostgreSQL served as the central repository for executing SQL queries and performing business analysis through **pgAdmin**.

Steps Performed

- Installed PostgreSQL libraries (psycopg2 and SQLAlchemy).
- Created a connection to the PostgreSQL database.
- Loaded the cleaned DataFrame into PostgreSQL using `to_sql()`.
- Verified successful data import.
- Accessed the database using **pgAdmin** to perform SQL analysis.

```
[17]: pip install psycopg2-binary sqlalchemy
Requirement already satisfied: psycopg2-binary in C:\Users\HP\AppData\Local\Programs\Python\Python313\Lib\site-packages (2.9.12)
Requirement already satisfied: sqlalchemy in C:\Users\HP\AppData\Local\Programs\Python\Python313\Lib\site-packages (2.0.51)
Requirement already satisfied: greenlet>=1 in C:\Users\HP\AppData\Local\Programs\Python\Python313\Lib\site-packages (from sqlalchemy) (3.5.3)
Requirement already satisfied: typing-extensions>=4.6.0 in C:\Users\HP\AppData\Local\Programs\Python\Python313\Lib\site-packages (from sqlalchemy) (4.15.0)
Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
[18]: import psycopg2
print(psycopg2.__version__)

2.9.12 (dt dec pq3 ext lo64)
```

```
[19]: import sqlalchemy
print(sqlalchemy.__version__)

2.0.51
```

```
[20]: !pip show psycopg2-binary

Name: psycopg2-binary
Version: 2.9.12
Summary: psycopg2 - Python-PostgreSQL Database Adapter
Home-page: https://psycopg.org/
Author: Federico Di Gregorio
Author-email: fog@initd.org
License: LGPL with exceptions
Location: C:\Users\HP\AppData\Local\Programs\Python\Python313\Lib\site-packages
Requires:
Required-by:
```

```
[21]: !pip show SQLAlchemy

Name: SQLAlchemy
Version: 2.0.51
Summary: Database Abstraction Library
Home-page: https://www.sqlalchemy.org
Author: Mike Bayer
Author-email: mike_mp@zzzcomputing.com
License: MIT
Location: C:\Users\HP\AppData\Local\Programs\Python\Python313\Lib\site-packages
Requires: greenlet, typing-extensions
Required-by:
```

```
[22]: from urllib.parse import quote_plus
      from sqlalchemy import create_engine

      #step 1: Connect to postgresSQL
      #Replace placeholders with ur actual details
      username = "postgres" #default
      password = "favaz@2294"
      host = "localhost"
      port = "5432"
      database = "customer_behavior"

      # URL-encode the password to handle special characters like '@'
      encoded_password = quote_plus(password)

      engine = create_engine(f"postgresql+psycopg2://{username}:{encoded_password}@{host}:{port}/{database}")

      #Step 2: Load database into postgresSQL
      table_name = "customer" #choose any table name
      df.to_sql(table_name, engine, if_exists="replace", index=False)

      print(f>Data successfully loaded into table '{table_name}' in database '{database}'.")

Data successfully loaded into table 'customer' in database 'customer_behavior'.
```

	customer_id bigint	age bigint	gender text	item_purchased text	category text	purchase_amount bigint	location text	size text	o te
1	1	55	Male	Blouse	Clothing	53	Kentucky	L	G
2	2	19	Male	Sweater	Clothing	64	Maine	L	M
3	3	50	Male	Jeans	Clothing	73	Massachuse...	S	M
4	4	21	Male	Sandals	Footwear	90	Rhode Island	M	M
5	5	45	Male	Blouse	Clothing	49	Oregon	M	T
6	6	46	Male	Sneakers	Footwear	20	Wyoming	M	V
7	7	63	Male	Shirt	Clothing	85	Montana	M	G
8	8	27	Male	Shorts	Clothing	34	Louisiana	L	C
9	9	26	Male	Coat	Outerwear	97	West Virginia	L	S
10	10	57	Male	Handbag	Accessori...	31	Missouri	M	F
11	11	53	Male	Shoes	Footwear	34	Arkansas	L	P
12	12	30	Male	Shorts	Clothing	68	Hawaii	S	C
13	13	61	Male	Coat	Outerwear	72	Delaware	M	G
14	14	65	Male	Dress	Clothing	51	New Hamps...	M	V
15	15	64	Male	Coat	Outerwear	53	New York	L	T
16	16	64	Male	Skirt	Clothing	81	Rhode Island	M	T
17	17	25	Male	Sunglasses	Accessori...	36	Alabama	S	G

Figure : Imported Dataset in PostgreSQL

The cleaned customer shopping behavior dataset was successfully imported into PostgreSQL and verified using pgAdmin before performing SQL-based business analysis

10. SQL Business Analysis

We performed structured analysis in PostgreSQL to answer key business questions:

1. Revenue by Gender – Compared total revenue generated by male vs. female customers.

```
--Q1. What is the total revenue generated by male vs. female customers?  
select gender, sum(purchase_amount) as revenue  
from customer  
group by gender
```

Output:

Showing rows: 1 to 2		Page No: 1	
	gender text	revenue numeric	
1	Female	75191	
2	Male	157890	

2. High-Spending Discount Users – Identified customers who used discounts but still spent above the average purchase amount.

```
--Q2. Which customers used a discount but still spent more than the average purchase amount?  
select customer_id, purchase_amount  
from customer  
where discount_applied = 'Yes' and purchase_amount >= (select AVG(purchase_Amount) from customer)
```

Output:

Data Output			
Messages			
Notifications			
	customer_id bigint	purchase_amount bigint	
1	2	64	
2	3	73	
3	4	90	
4	7	85	
5	9	97	
6	12	68	
7	13	72	
8	16	81	
9	20	90	
10	22	62	

3. Top 5 Products by Rating – Found products with the highest average review ratings.

```
--Q3. Which are the top 5 products with the highest average review rating?  
select item_purchased, AVG(review_rating) as "average review rating"  
from customer  
group by item_purchased  
order by avg(review_rating) desc  
limit 5
```

Output:

Data Output Messages Notifications		
	item_purchased text	average review rating double precision
1	Gloves	3.8614285714285725
2	Sandals	3.8443750000000003
3	Boots	3.8187500000000005
4	Hat	3.8012987012987005
5	Skirt	3.784810126582278

4. Shipping Type Comparison – Compared average purchase amounts between Standard and Express shipping.

```
--Q4. Compare the average Purchase Amounts between Standard and Express Shipping.  
select shipping_type, ROUND(AVG(purchase_amount), 2)  
from customer  
where shipping_type in ('Standard', 'Express')  
group by shipping_type
```

Output:

Data Output Messages Notifications		
	shipping_type text	round numeric
1	Standard	58.46
2	Express	60.48

5. Subscribers vs. Non-Subscribers – Compared average spend and total revenue across subscription status.

```
--Q5. Do subscribed customers spend more? Compare average spend and total revenue
--between subscribers and non-subscribers.

select subscription_status,
COUNT(customer_id) as total_customers,
ROUND(AVG(purchase_amount),2) as avg_spend,
ROUND(SUM(purchase_amount),2) as total_revenue
from customer
group by subscription_status
order by total_revenue,avg_spend desc;
```

Output:

	subscription_status text	total_customers bigint	avg_spend numeric	total_revenue numeric
1	Yes	1053	59.49	62645.00
2	No	2847	59.87	170436.00

6. Discount-Dependent Products – Identified 5 products with the highest percentage of discounted purchases.

```
--Q6. Which 5 products have the highest percentage of purchases with discounts applied?
select item_purchased,
ROUND(100 * SUM(CASE WHEN discount_applied = 'Yes' THEN 1 ELSE 0 END)/COUNT(*),2) AS discount_rate
from customer
group by item_purchased
order by discount_rate desc
limit 5;
```

Output:

	item_purchased text	discount_rate numeric
1	Hat	50.00
2	Sneakers	49.00
3	Coat	49.00
4	Sweater	48.00
5	Pants	47.00

7. Customer Segmentation – Classified customers into New, Returning, and Loyal segments based on purchase history.

```
--Q7. Segment customers into New, Returning, and Loyal based on their total
-- number of previous purchases, and show the count of each segment.
with customer_type as (
select customer_id,previous_purchases,
Case
    WHEN previous_purchases = 1 THEN 'new'
    WHEN previous_purchases BETWEEN 2 and 10 THEN 'Returning'
    ELSE 'Loyal'
END AS customer_segment
from customer
)
select customer_segment,count(*) as "Number of customers"
from customer_type
group by customer_segment
```

Output:

	customer_segment text	Number of customers bigint
1	Loyal	3116
2	new	83
3	Returning	701

8. Top 3 Products per Category – Listed the most purchased products within each category.

```
--Q8. What are the top 3 most purchased products within each category?
WITH item_counts AS (
    SELECT category,
           item_purchased,
           COUNT(customer_id) AS total_orders,
           ROW_NUMBER() OVER (PARTITION BY category ORDER BY COUNT(customer_id) DESC) AS item_rank
    FROM customer
    GROUP BY category, item_purchased
)
SELECT item_rank,category, item_purchased, total_orders
FROM item_counts
WHERE item_rank <=3;
```

Output:

	item_rank bigint	category text	item_purchased text	total_orders bigint
1	1	Accessori...	Jewelry	171
2	2	Accessori...	Sunglasses	161
3	3	Accessori...	Belt	161
4	1	Clothing	Blouse	171
5	2	Clothing	Pants	171
6	3	Clothing	Shirt	169
7	1	Footwear	Sandals	160
8	2	Footwear	Shoes	150
9	3	Footwear	Sneakers	145
10	1	Outerwear	Jacket	163

9. Repeat Buyers & Subscriptions – Checked whether customers with >5 purchases are more likely to subscribe.

```
--Q9. Are customers who are repeat buyers (more than 5 previous purchases) also likely to subscribe?  
SELECT subscription_status,  
       COUNT(customer_id) AS repeat_buyers  
FROM customer  
WHERE previous_purchases > 5  
GROUP BY subscription_status;
```

Output:

Data Output			Messages	Notifications
	subscription_status text	repeat_buyers bigint	SQL query of data	
1	No	2518		
2	Yes	958		

10. Revenue by Age Group – Calculated total revenue contribution of each age group.

```
--Q10. What is the revenue contribution of each age group?  
SELECT  
       age_group,  
       SUM(purchase_amount) AS total_revenue  
FROM customer  
GROUP BY age_group  
ORDER BY total_revenue desc;
```

Output:

Data Output			Messages	Notifications
	age_group text	total_revenue numeric		
1	Young Adult	62143		
2	Middle-aged	59197		
3	Adult	55978		
4	Senior	55763		

11.RFM-Style Customer Segmentation

To extend the customer behavior analysis beyond descriptive and SQL-based insights, an RFM-style customer segmentation approach was implemented. RFM analysis evaluates customers based on Recency, Frequency, and Monetary value to identify differences in customer engagement and value.

The dataset does not contain transaction dates or last-purchase timestamps. Therefore, true Recency could not be calculated. Instead, the engineered `purchase_frequency_days` feature was used as a documented Recency proxy. Lower values indicate more frequent purchasing behavior and therefore receive higher Recency scores.

RFM Components:

Component	Feature Used	Interpretation
Recency (Proxy)	<code>purchase_frequency_days</code>	Estimated days between purchases; lower values indicate more frequent purchasing
Frequency	<code>previous_purchases</code>	Number of previous purchases made by the customer
Monetary	<code>purchase_amount</code>	Purchase amount associated with the customer

RFM Scoring and Segmentation:

Each RFM component was divided into five groups using PostgreSQL `NTILE(5)` window functions. Recency was scored in descending order of estimated days so that customers with shorter purchase intervals received higher Recency scores. Frequency and Monetary values were scored in ascending order so that customers with higher purchase counts and spending received higher scores.

The three component scores were combined to produce an overall RFM score ranging from 3 to 15. Customers were then classified into five actionable segments based on their total score.

Customer Segment Classification:

RFM Score	Customer Segment
≥ 13	Champions
≥ 10	Loyal Customers
≥ 7	Potential Loyalists
≥ 5	At Risk
< 5	Low Value / Churn Risk

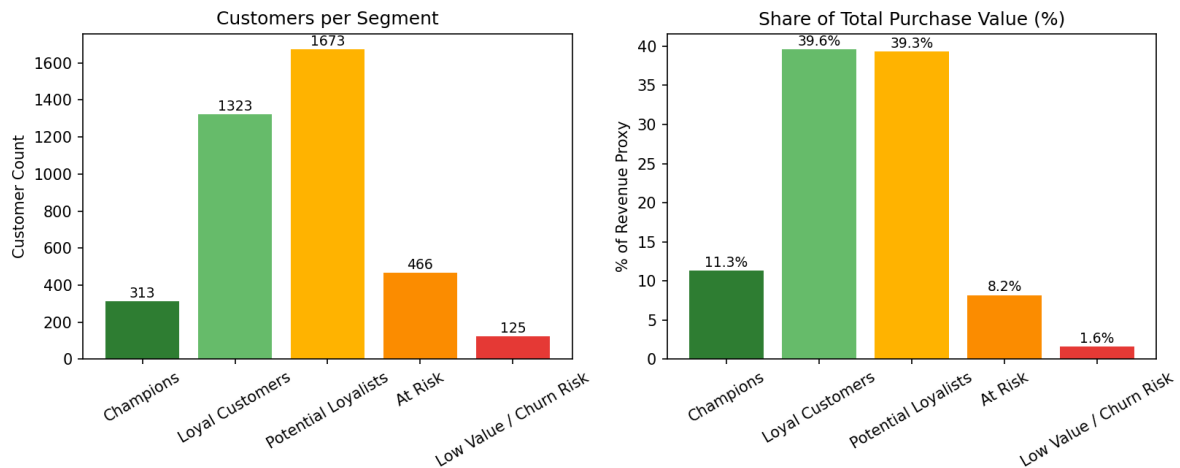


Figure 11.1: RFM-Style Customer Segment Distribution and Share of Total Purchase Value

The RFM-style segmentation classified all 3,900 customers into five segments. Potential Loyalists formed the largest segment with 1,673 customers (42.9%), while Loyal Customers accounted for 1,323 customers (33.9%). Although Champions represented only 8.0% of customers, they had the highest average purchase amount of \$83.97. Loyal Customers contributed the largest share of total purchase value at 39.6%, closely followed by Potential Loyalists at 39.3%.

RFM Segmentation Results

Segment	Customers	Customer Share	Avg. Purchase Amount	Purchase Value Share
Champions	313	8.0%	\$83.97	11.3%
Loyal Customers	1,323	33.9%	\$69.76	39.6%
Potential Loyalists	1,673	42.9%	\$54.71	39.3%
At Risk	466	11.9%	\$41.04	8.2%
Low Value / Churn Risk	125	3.2%	\$30.75	1.6%
Total	3,900	100%	—	100%

The results show that Loyal Customers and Potential Loyalists together represent 76.8% of the customer base and approximately 78.9% of the total purchase value. This indicates that retention and conversion strategies focused on these two groups could have a significant business impact. Champions, although smaller in number, demonstrate the highest average purchase amount and represent a valuable customer group for loyalty and retention initiatives.

Implementation:

The segmentation was implemented in both Python and PostgreSQL. Python generated customer-level RFM scores, segment assignments, summary statistics, and visualizations. PostgreSQL reproduced the logic using CTEs, NTILE(5), aggregate functions, and CASE statements. The outputs were exported for further analysis.

12. Dashboard in Power BI

Finally, we built an interactive dashboard in Power BI to present insights visually.



Figure 11.1: Customer Behavior Dashboard in Power BI

After completing the SQL business analysis and RFM-style customer segmentation, an interactive dashboard was developed in Power BI to visualize key business metrics and customer shopping trends. The dashboard enables users to monitor customer behavior, analyse sales performance, and filter insights dynamically using interactive slicers.

The dashboard presents key performance indicators such as the total number of customers, average purchase amount, and average review rating. It also includes visualizations for subscription status, revenue by category, sales by category, revenue by age group, and sales by age group. Interactive filters based on subscription status, gender, product category, and shipping type allow users to explore the data from different perspectives and gain meaningful business insights.

13. Key Insights

- Clothing generated the highest revenue among all product categories.
- Young Adult customers contributed the highest overall revenue.
- The majority of customers were non-subscribers, indicating an opportunity to increase subscription adoption.
- Express shipping customers recorded a slightly higher average purchase amount than standard shipping customers.
- RFM-style segmentation identified Potential Loyalists as the largest customer segment, comprising 42.9% of customers, while Loyal Customers contributed the largest share of total purchase value at 39.6%.
- Potential Loyalists and Loyal Customers together represented 76.8% of customers and approximately 78.9% of total purchase value, highlighting significant opportunities for customer retention and conversion strategies.

14. Business Recommendations

- Introduce targeted subscription offers to encourage more customers to join the subscription program.
- Focus marketing campaigns on high-performing product categories to maximise revenue.
- Develop targeted retention and loyalty campaigns for **Champions and Loyal Customers** to maintain their high purchasing value.
- Target **Potential Loyalists** with personalised offers and engagement campaigns to encourage them to become Loyal Customers.
- Develop re-engagement campaigns for **At Risk** customers using targeted discounts, reminders, or personalised product recommendations to reduce potential churn.
- Promote highly rated products to improve customer engagement and sales.

15. Conclusion

This project successfully demonstrated an end-to-end retail data analytics workflow using Python, PostgreSQL, SQL, RFM-style customer segmentation, and Power BI. The dataset was preprocessed and feature-engineered using Python, stored in a relational PostgreSQL database, and analysed using SQL to answer real-world business questions. RFM-style segmentation further extended the analysis by classifying customers according to purchasing frequency, purchase history, and purchase value, while accounting for the absence of transaction timestamps through a documented Recency proxy. The resulting customer segments provided additional insights into customer value, retention opportunities, and potential churn risk. Finally, the findings were presented through an interactive Power BI dashboard to support data-driven business decisions.